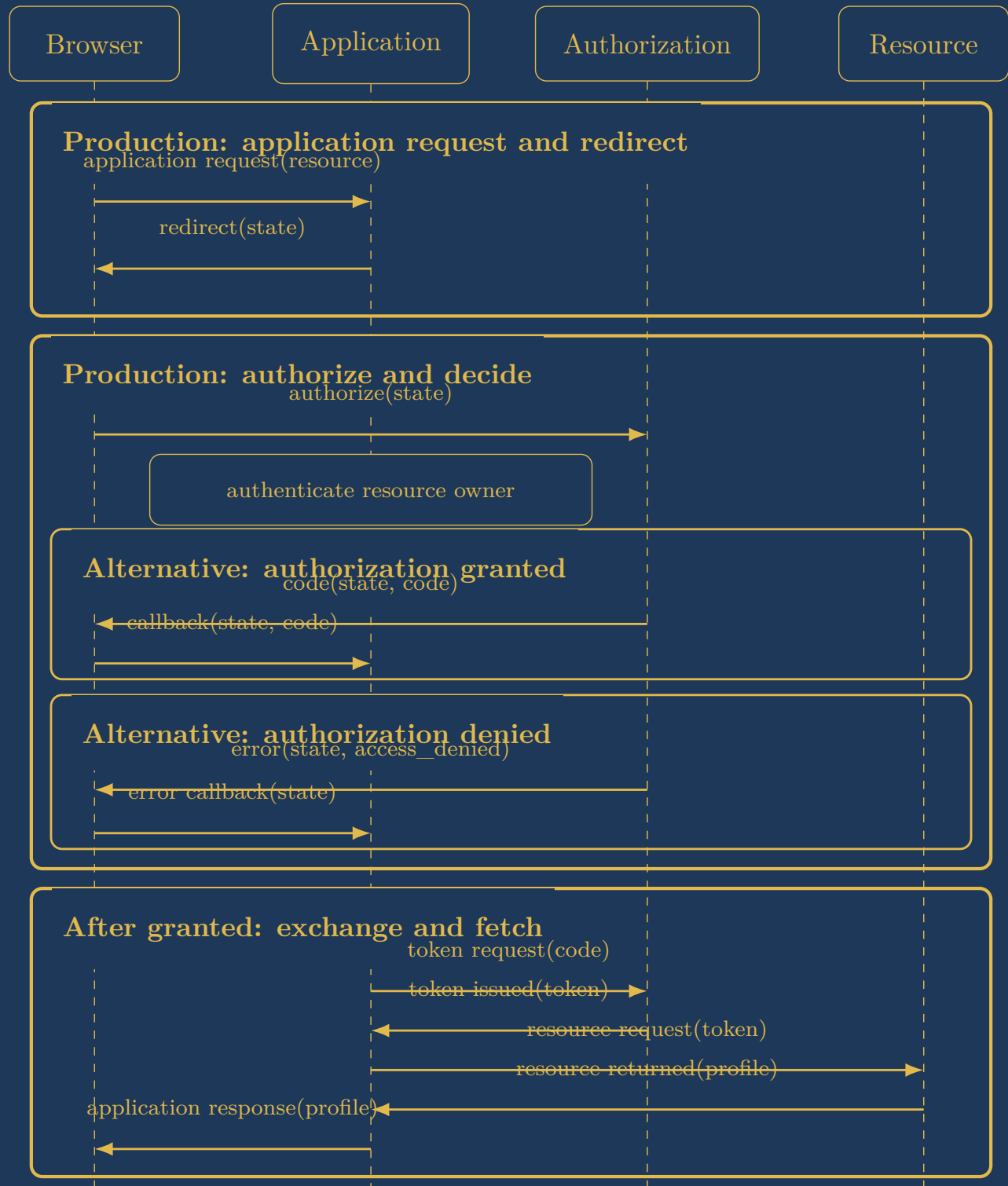


Grammar-based Formal Methods

Vibecoding

OAuth authorization-code flow



Preface

Formal methods should be the easy way to build software. This book develops that claim into a practical method: describe externally meaningful behaviour as a parameterised, multiparty grammar; interpret its sentences as event traces; state temporal requirements in CTL; and render the whole specification as ordinary Lean. The specification says what participants may observe, not how an implementation stores every intermediate detail.

This is a working book and an executable design notebook. Definitions will migrate into a stable Lean library; systems built with the library should consist mostly of domain types, grammar productions, and theorems about expected behaviour. The evolving implementation, examples, and book sources are maintained in the LeanFM repository [2].

Contents

Preface	i
I The Grammar-Based Method	1
1 Formal Methods as the Default Path	2
1.1 Behaviour before machinery	2
1.2 Three ingredients	2
1.3 A test for every abstraction	3
2 Histories Made of Events	4
2.1 Derived state	4
2.2 Time is more than a timestamp	5
3 A Multiparty Grammar of Behaviour	6
3.1 A small authoring algebra	6
3.2 Parameters carry information	6
3.3 Composition without omniscience	7
3.4 Actors multiplex bounded conversations	7
4 The Scenario Authoring Language	8
4.1 The smallest useful scenario	8
4.1.1 Variables and assignment	9
4.2 Vague messages first, typed alphabets later	9
4.3 One concern per scenario	10
4.4 Interaction diagram: accepted quote	10
4.5 Choices and unresolved probability	10
4.6 Specified dwell time	11
4.7 Estimating completion from an event stream	12
4.8 Composing requirements means intersecting constraints	12
4.9 State-machine rendering	13
4.10 Charts are declarative trace reductions	13
5 Worked Example: OAuth Authorization Code Flow	17
5.1 Actors, values, and observable events	17
5.2 Production 1: application request and redirect	18

5.3	Production 2: authorize, then grant or deny	19
5.4	Scenario 3: exchange the code and use the token	20
5.5	Reading the scenarios together	20
5.6	Interaction diagrams by scenario	21
5.7	Requirements as observable constraints	21
5.7.1	Password secrecy	21
5.7.2	Byte-level refinement without choosing a transport	23
5.8	Probabilities and charts	23
5.9	Residual state machine	24
6	CTL over Generated Futures	26
6.1	Past claims	26
6.2	Fairness and time bounds	26
7	Lean as the Rendered Language	28
7.1	From specification to service	28
8	Research and Implementation Roadmap	29
8.1	Milestones	29
8.2	Proof obligations	29
II	Security Protocol Exercises from the CSP Approach	31
9	From CSP Roles to Multiparty Grammars	32
9.1	Can actors really be specified separately?	32
9.1.1	Local state without global implementation state	33
9.2	A transport-neutral byte algebra	33
9.3	Yahalom as three separate actors	33
9.3.1	Alice, the initiator	33
9.3.2	Bob, the responder	34
9.3.3	The trusted server	34
9.3.4	Global assembly and context conditions	35
9.4	The attacker is another grammar plus deduction	35
9.5	Secrecy and agreement become theorems	36
9.6	Otway–Rees: guards are not optional decoration	36
9.7	Needham–Schroeder: public-key lookup as behaviour	37
9.8	What these exercises demand from the library	37
	Bibliography	39
	Index	40

Part I

The Grammar-Based Method

Chapter 1

Formal Methods as the Default Path

Formal methods are usually presented as an extra stage: first build a system, then translate a simplified version into a model, and finally ask a specialist tool a few questions. That workflow taxes correctness. The model drifts, the difficult translation is postponed, and proofs concern yesterday's architecture.

Our aim is the reverse. The shortest path to a running service should pass through an executable behavioural specification. Writing the specification should remove work: it supplies message types, scenario tests, model-checking input, documentation, and the skeleton of the service itself. Going without it should be the inconvenient choice.

1.1 Behaviour before machinery

The primary object is a set of permitted, visible histories. An implementation may use threads, actors, queues, databases, caches, or continuations. Those decisions matter operationally, but they are not automatically part of the contract. Promoting all internal state into the model creates a product state space whose size grows exponentially with independently varying components. Worse, it makes the specification brittle under harmless refactoring.

We therefore begin with *behaviour*: who communicated with whom, what kind of event occurred, what information it carried, and what causal history it extends. Internal state enters only when it changes an observable promise.

1.2 Three ingredients

The method combines three traditions.

1. Language theory gives compact generative descriptions of legal histories. Regular and context-free grammars provide the approachable starting point. Dana Richards and Henry Hamburger, the author's college professors, are cited in Richards and Hamburger [4] as the source from which the author learned these methods, not as their originators. For the underlying habits of symbolization, validity, proof, quantification, and modal reasoning, we also draw on the accessible progression of Gensler [3].
2. Protocol analysis contributes explicit participants, adversarial choices, trace semantics, and tool-supported checking. The CSP treatment of Ryan et al. [5], including Steve

Schneider, is cited as the source from which the author learned this treatment.

3. Temporal logic turns expectations into queries over the generated transition system. CTL is central because it distinguishes claims about some possible future from claims about every possible future [1].

1.3 A test for every abstraction

Each abstraction in this book should answer a practical question:

Can a developer state a meaningful behaviour once, compose it with other behaviours, check its consequences, and keep it stable while implementation details change?

If an abstraction does not improve that workflow, it does not belong in the core library.

Chapter 2

Histories Made of Events

Every execution is represented by an event sequence. A deliberately small common envelope makes histories comparable across protocols:

```
structure Event (Actor Session Task Kind Payload : Type) where
  clock    : ClockTimestamp
  id       : String
  priorIds : List String
  session  : Session
  task     : Task
  from     : Actor
  to       : Actor
  kind     : Kind
  payload  : Payload

abbrev Trace (Actor Session Task Kind Payload : Type) :=
  List (Event Actor Session Task Kind Payload)
```

The fields have different jobs. `id` identifies a visible fact; `priorIds` records zero or more immediate causal claims; `from` and `to` name the parties; `(session, task)` correlates independently progressing conversations; and `kind` supplies a stable discriminator. The payload is protocol-specific and carries the parameters that flow from one production to the next. `clock` is sampled from the model's shared monotonic clock. Its differences measure elapsed time; it is not a civil-time date and its origin is immaterial.

The causal links form a directed acyclic graph rather than a singly linked list. An initial event has no prior identifiers. A fork is represented by several later events that each name the same prior event. A join is represented by one event whose `priorIds` names every branch on which it immediately depends.

2.1 Derived state

A current session, an authenticated principal, or an outstanding request can often be computed by folding the trace. Such a fact need not also exist as primitive model state. This discipline has two benefits: evidence for a fact remains visible, and the state representation does not multiply independently of the history that justifies it.

Some implementations necessarily maintain private state. The model includes it only when two executions with the same visible trace may make different future behaviour legal. Even then, the first remedy is to expose the relevant distinction as an event or a grammar parameter rather than copying an implementation object graph into the specification.

2.2 Time is more than a timestamp

The central modelling problem is maintainable time evolution. We separate three ideas that are often conflated:

- sequence order: one visible event occurs before another;
- causality: an event extends or responds to another event;
- duration: an amount of physical or logical time elapses.

Grammar structure handles sequence, identifiers handle causal linkage, and typed constraints handle duration. This keeps a change in timeout policy from rewriting the protocol's entire behavioural shape.

For two events from the same `(session, task)`, elapsed time is defined when the first event's clock value does not exceed the second's:

```
def elapsed (start finish : Event) : Option Duration :=
  if start.session = finish.session &&
    start.task = finish.task &&
    start.clock <= finish.clock
  then some (finish.clock - start.clock)
  else none
```

Latency queries name their boundary event kinds and correlate them by `(session, task)` before subtracting clock values. Causality still comes from grammar structure and `priorIds`: equal timestamps may occur, and a clock value alone does not establish that one event caused another.

Chapter 3

A Multiparty Grammar of Behaviour

A protocol is a language whose terminal symbols are visible events. A production describes a legal continuation. The context-free grammar is the visual and conceptual starting point, not a claim that every useful protocol language is context-free.

3.1 A small authoring algebra

```
inductive Grammar (Event : Type) where
| empty
| event  : Event -> Grammar Event
| seq    : Grammar Event -> Grammar Event -> Grammar Event
| choice : List (Grammar Event) -> Grammar Event
| par    : Grammar Event -> Grammar Event -> Grammar Event
| where_ : (List Event -> Bool) -> Grammar Event -> Grammar Event
```

`seq` preserves time order. `choice` exposes alternatives rather than burying them in control flow. `par` admits interleavings while preserving the internal order of each side. A `where_` clause checks a fact accumulated from the prior trace and provides the escape from context-free structure when genuine context sensitivity is needed.

Simple finite conversations can compile to regular languages. Nested request structure motivates context-free productions. Correlation, freshness, uniqueness, and authorization typically require parameter flow or guarded productions.

3.2 Parameters carry information

A production should bind values at one event and reuse them later. Conceptually:

```
def login (browser server provider : Actor) (state : OAuthState) : Grammar Event :=
  event (redirect server browser provider state) >>>
  event (authorize browser provider state) >>>
  callback browser server state
```

The repeated `state` is not decorative. It states the information-flow relationship that distinguishes a matching callback from an unrelated one. Lean's types, constructors, and ordinary function parameters are the surface language; the fixed library supplies grammar composition and semantics.

3.3 Composition without omniscience

A participant’s implementation need not know the global execution state. The global grammar states the contract, while projection derives each participant’s view. Composition should preserve independently specified modules and make shared events explicit. Proving useful projection and composition theorems is a core research obligation, not an implementation footnote.

3.4 Actors multiplex bounded conversations

An actor has separate finite-capacity inbound and outbound message queues. Queue capacity is part of the observable execution model: producing a message may block when the outbound queue is full, and transport may block when the destination’s inbound queue is full. Sending is therefore distinct from delivery and from the destination actor consuming the message.

Messages carry a session identifier as well as a task identifier. Their pair is the correlation key for actor-local work:

```

structure WorkKey where
  session : SessionId
  task    : TaskId

structure ActorRuntime where
  inboundCapacity  : Nat
  outboundCapacity : Nat
  inbound          : List Envelope
  outbound         : List Envelope
  inFlight         : List (Prod WorkKey GrammarResidual)
  inboundBounded   : inbound.length <= inboundCapacity
  outboundBounded  : outbound.length <= outboundCapacity

```

An actor may have zero or many distinct `WorkKeys` in flight. Consuming an inbound envelope selects or creates the residual associated with that envelope’s `(session, task)` pair; it does not replace one actor-wide “current task.” Messages from different keys may interleave while the grammar preserves the order required within each conversation.

The queue transitions are explicit:

- an actor step appends a produced envelope to its outbound queue;
- a transport step moves the outbound head to the destination’s inbound queue when both operations are within capacity;
- a receive step removes an inbound envelope and advances the matching `(session, task)` residual;
- a full outbound queue blocks production, a full inbound queue blocks transport, and an empty inbound queue blocks receive.

Queue order and causal order are not identical. FIFO determines which queued envelope is available next, while `priorIds` records the causal graph among events, including forks and joins across actors.

Chapter 4

The Scenario Authoring Language

This chapter proposes the first concrete surface language. It is intentionally a small Lean DSL: ordinary `def`, function parameters, `let`, typed values, and `do` notation provide the host language; a fixed library adds scenario, endpoint, choice, probability, and rendering operations. The examples are a design target. Names may change as the Lean implementation makes their semantics precise.

4.1 The smallest useful scenario

A scenario is a named grammar fragment. Every emitted terminal is an observable message event. Endpoint commands establish the `from` and `to` context for the messages that follow.

```
def requestQuote (customer shop : Actor) : Scenario :=
  scenario "request a quote" do
    let requestId <- declare RequestId "requestId"

    from_ customer
    to_ shop
    emit "quote requested" [field "requestId" requestId]

    let quoteId <- declare QuoteId "quoteId"
    let amount <- declare Money "amount"

    from_ shop
    to_ customer
    emit "quote offered"
      [ field "requestId" requestId
        , field "quoteId" quoteId
        , field "amount" amount
      ]
```

The spelling `from_` avoids collision with Lean syntax while remaining visually explicit. Endpoint commands affect authoring context only; elaboration copies both endpoints into every event. A scenario with an `emit` before both endpoints have been established is rejected.

4.1.1 Variables and assignment

There are two kinds of assignment.

- Ordinary `let x := expression` evaluates a known Lean expression.
- `let x <- declare T "x"` introduces a symbolic value of type `T`. The grammar binds it when generating or matching a trace, and later events may reuse it.

Variables are immutable. A later `let attempt := attempt + 1` creates a new binding rather than mutable hidden state. Information flow is consequently visible in the grammar's data dependencies. Equality of two occurrences of `requestId` means they carry the same value, not merely values with the same field name.

A value received from the environment is bound by matching an observable event:

```
from_ customer
to_ shop
receive "quote accepted" do
  let quoteId <- take QuoteId "quoteId"
  let payment <- take PaymentToken "paymentToken"

from_ shop
to_ bank
emit "payment authorization requested"
  [field "quoteId" quoteId, field "paymentToken" payment]
```

`receive` and `emit` both denote grammar terminals; they differ only in the viewpoint used when projecting a global scenario to executable participant code.

4.2 Vague messages first, typed alphabets later

Early requirements should not require a finished protobuf schema. A vague message has a stable semantic name and a list of named, typed fields. It is already precise enough to express ordering, correlation, and temporal properties.

```
emit "quote offered"
  [field "requestId" requestId, field "amount" amount]
```

Later, a refinement maps that terminal to a concrete alphabet:

```
refineMessage "quote offered" with Proto.Shop.QuoteOffered where
  request_id := field "requestId"
  quote_id   := field "quoteId"
  cents      := encodeMoney (field "amount")
```

The refinement obligation is language inclusion: every concrete implementation trace, after erasing representation details, must be admitted by the abstract grammar. Refinement may decide serialization, retry mechanics, and internal state; it must not invent externally visible behaviour forbidden by the requirements.

4.3 One concern per scenario

Scenarios should be small enough to read as stories and independent enough to render separately. The quote protocol can be decomposed as follows.

```
def quoteAccepted (customer shop : Actor) : Scenario :=
  scenario "quote accepted" do
    include (requestQuote customer shop)
    -- bind quoteId from the included scenario's exports
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote accepted" [field "quoteId" quoteId]

def quoteDeclined (customer shop : Actor) : Scenario :=
  scenario "quote declined" do
    include (requestQuote customer shop)
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote declined" [field "quoteId" quoteId]

def quoteOutcome (customer shop : Actor) : Scenario :=
  oneOf "customer decision"
  [ quoteAccepted customer shop
    , quoteDeclined customer shop
  ]
```

`include` is textual only in the rendered book view; semantically it is a reference to a named nonterminal. The compiler reads all referenced scenarios, checks exported bindings, detects cycles, and builds one grammar. Tools retain the scenario boundary for navigation, diagrams, tests, and counterexamples.

4.4 Interaction diagram: accepted quote

The first rendering of any scenario is a sequence-style interaction diagram. It shows observable messages and parameter flow without suggesting an internal implementation architecture.

4.5 Choices and unresolved probability

Nondeterminism and probability are different. `oneOf` says that multiple behaviours are legal without claiming how often they occur. `weighted` attaches a distribution when the abstraction has one.

```
def paymentOutcome : Scenario :=
  oneOf "payment outcome" [paymentApproved, paymentRejected]

def estimatedCustomerDecision : Scenario :=
  choose customer "customer decision"
  [ weighted (95 / 100) customerSubmitsPayment
    , weighted ( 5 / 100) customerAbandonsPayment
  ]
```

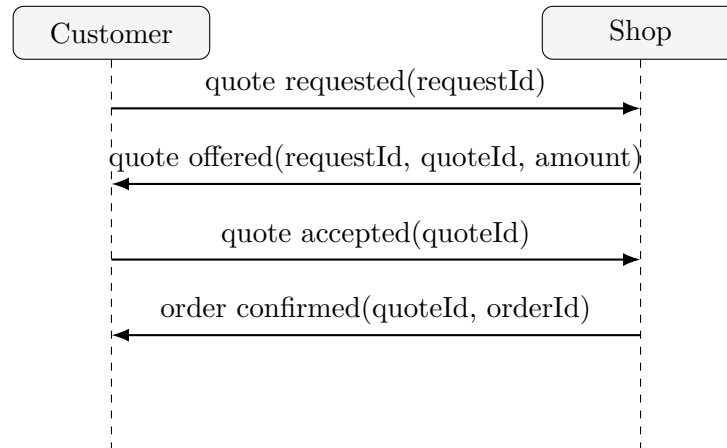


Figure 4.1: Interaction diagram derived from the accepted-quote scenario.

Probabilities belong to a choice source, not to messages. A deterministic service that consumes probabilistic user input remains part of the induced probabilistic system: its probability-one transitions carry the incoming path mass forward. An early requirement can use `oneOf`, a symbolic weight, or a range such as `between 0.90 0.99`. Implementation refinement resolves those claims to a well-defined stochastic model or to deterministic conditions. Refinement must preserve possibility even when a current implementation assigns probability zero to a legal abstract branch; otherwise changing operational policy would silently change the contract.

4.6 Specified dwell time

A specification may attach a dwell time to an event step or to a named production. The dwell is a duration on the same monotonic time scale as event clocks:

```

emit "token requested" [field "code" code]
  |>.dwell 3.milliseconds

include exchangeAndFetch
  |>.dwell 12.milliseconds
  
```

Taking a step with dwell d advances the model clock from t to $t + d$ before timestamping the event that completes the step. A production dwell constrains the elapsed time between its entry and exit boundary events; it does not assign the same dwell independently to every contained event. Exact values, bounded ranges, or weighted dwell outcomes may be used when the corresponding timing information is available.

Dwell is part of the specification, so it can drive latency, throughput, timeout, and queue-pressure calculations. It does not create causality: the grammar and `priorIds` continue to determine which events depend on which others.

4.7 Estimating completion from an event stream

The same specification can consume observed events. The reducer first partitions the stream by `(session, task)`, orders each partition by its monotonic clock timestamp, and matches the production’s declared entry and terminal event kinds. Each partition then yields a task attempt:

```
structure TaskAttempt where
  key          : WorkKey
  startedAt    : ClockTimestamp
  completedAt  : Option ClockTimestamp
  outcome      : Option TaskOutcome

def latency (attempt : TaskAttempt) : Option Duration :=
  attempt.completedAt.map fun finish =>
    finish - attempt.startedAt
```

For an observation window containing N starts, C matched terminal events, S successful terminals, and completed latencies $L_1, \dots, L_C$, the basic estimates are

$$\hat{P}(\text{complete}) = \frac{C}{N}, \quad \hat{P}(\text{success} \mid \text{complete}) = \frac{S}{C}, \quad \hat{E}[L \mid \text{complete}] = \frac{\sum_i L_i}{C}.$$

Attempts without a terminal event remain *in flight*. They contribute to the number started and to queue/load measurements, but not to completed-task latency or to the conditional success denominator. Calling them failures would bias both probability and latency whenever the observation window ends before they do.

The reducer can group these sufficient statistics by task, actor, time bucket, maximum concurrency, or queue occupancy and feed them directly to pie, time-series, and XY plots. Specified probabilities and dwell times are therefore predictions; stream-derived estimates provide measurements against which those predictions can be checked.

In the executable reducer, an entry event opens a `TaskAttempt`, progress events leave it open, and a success or failure event closes the open attempt with the same `(session, task)` key. The resulting attempts feed the estimates above.

4.8 Composing requirements means intersecting constraints

Scenario composition builds behaviour in familiar language-theoretic ways:

Operator	Language operation	Purpose
<code>then a b</code>	concatenation	a happens before b
<code>oneOf [a,b]</code>	union	either story is legal
<code>together a b</code>	shuffle/interleaving	independent concurrent stories
<code>repeat a</code>	closure	repeated conversation
<code>where_ p a</code>	trace filtering	context-sensitive legality

Combining requirements is subtler. A requirement usually does not add another message sequence; it removes histories that violate a concern. Therefore `requireAll` denotes intersection of accepted trace sets.

```

def quoteSystem : SystemSpec :=
  generatedBy quoteOutcome
  |> requireAll
    [ correlate "offered quote matches request"
      (sameField "requestId")
    , forbid "confirmation without acceptance"
      (event "order confirmed" && notPreviously "quote accepted")
    , ctl "every accepted quote is answered"
      (AG (accepted --> AF (confirmed || rejected)))
    , timing "answer within service objective"
      (accepted --> within 2.seconds (confirmed || rejected))
    ]

```

This distinction prevents an important category error: union and concatenation assemble possible stories; intersection layers independent obligations over those stories. A system is consistent only if the intersection remains nonempty. When it is empty, tooling should return a minimal conflicting set of requirements and the shortest available counterexample prefix.

4.9 State-machine rendering

The same scenario grammar can be rendered as a state machine. States are grammar residuals—the legal continuations after a trace prefix—rather than snapshots of private implementation memory.

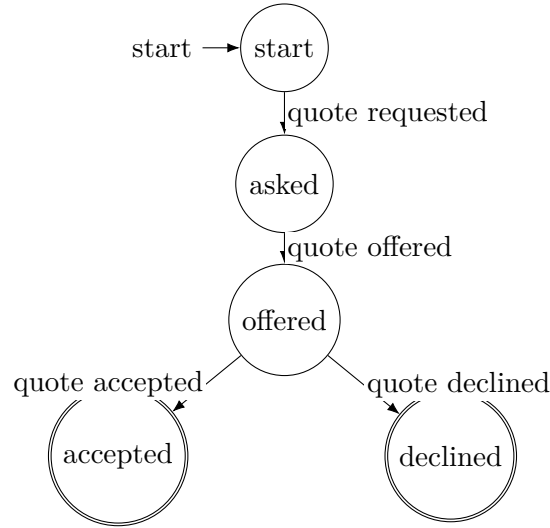


Figure 4.2: A state machine derived from grammar residuals.

4.10 Charts are declarative trace reductions

Charts are also specified in the Lean subset. A chart names an observable event source, a grouping or x expression, and an aggregate or y expression. It does not contain rendering code.

```

def outcomeShare : ChartSpec :=
  pie "quote outcomes" do
    source events
    where_ (kind == "quote accepted" || kind == "quote declined")
    groupBy kind
    value count

def responseLatency : ChartSpec :=
  line "quote response latency" do
    source (correlate events by "quoteId")
    x clock
    y (elapsed "quote offered" "customer decision")
    series "p50" median

def latencyByConcurrency : ChartSpec :=
  xyplot "latency by concurrent sessions" do
    source completedRuns
    x maxConcurrentSessions
    y (mean (elapsed firstEvent terminalEvent))

def queueByOfferedLoad : ChartSpec :=
  xyplot "queue occupancy by offered load" do
    source completedRuns
    x requestsPerSecond
    y (mean totalQueueOccupancy)

def propagatedGrantProbability : ChartSpec :=
  xyplot "user probability propagated through deterministic servers" do
    source parameterSweep
    x userGrantProbability
    y terminalGrantProbability

```

An XY plot preserves two numeric axes. It differs from a time series, whose x -axis is specifically an event clock, and from a categorical chart, whose x values are labels. The first two examples reduce each completed model run to one point. The third sweeps a model parameter and checks that deterministic downstream processing transports the user's probability mass.

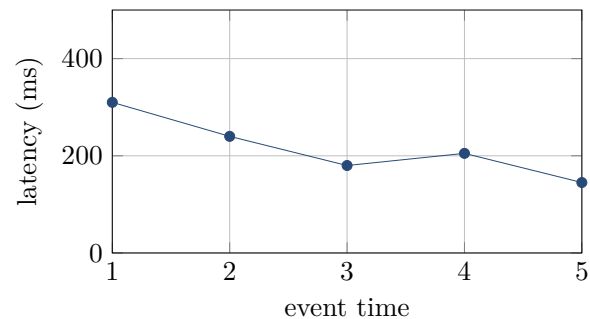
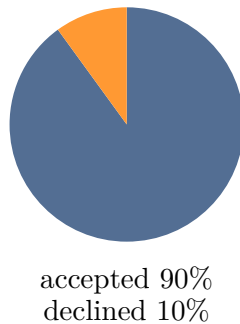


Figure 4.3: Example pie and line renderings from declarative chart specifications.

Interaction diagrams, state machines, and charts are projections of the same typed scenario

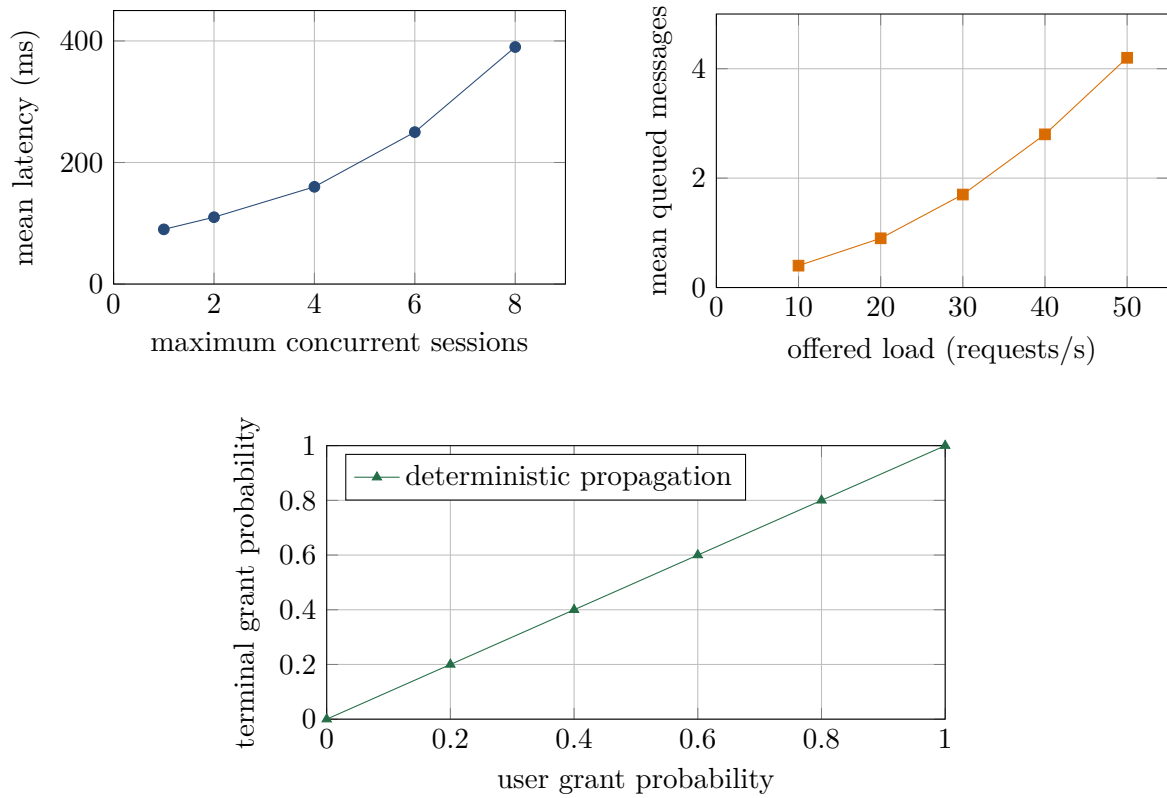


Figure 4.4: XY-plot examples. The values are illustrative model-run data: clocks and correlation produce latency, bounded queues produce occupancy, and probability-one server steps preserve user-originated grant mass.

and event definitions. Keeping them derived prevents a diagram from becoming a second, drifting specification.

Chapter 5

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow. It is not a replacement for the OAuth specification and deliberately begins above HTTP and protobuf. Its purpose is to exercise scenario decomposition, parameter flow, visible evidence, alternatives, requirements, and rendering.

5.1 Actors, values, and observable events

The example has four participants: a person using a browser, an application, an authorization service, and a resource service. The browser is modelled separately from the person because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State      : Type
opaque AuthCode   : Type
opaque AccessToken : Type
opaque Resource   : Type
opaque Profile    : Type
opaque Password   : Type
opaque Bytes      : Type
```

Every terminal elaborates to the common event envelope. For example, the vague message “authorization redirected” becomes conceptually:

```
{ clock := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

Before the redirect returns, the authorization service may authenticate the person. OAuth does not prescribe the authentication mechanism; we include a password branch to exercise secret input without pretending that the password is an OAuth protocol parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
    (protectFor .authorizationServer
      (encode [secretField "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
    [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
    ]
```

The wire bytes are specified even though the transport is not. The abstract `protectFor` constructor promises confidentiality to the named endpoint. A later refinement may realize that promise with TLS, message-level encryption, or a local trusted channel. Merely placing the password inside a field would not support a secrecy theorem: a network observer would then derive it from the bytes.

5.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

This production treats the application request and its redirect as one named unit. It establishes the correlation value `state` and exports it to the next production. Nothing here says how the application stores it.

5.3 Production 2: authorize, then grant or deny

```

def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
    [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
    [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
    [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
    [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
    [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
    [ include authorizationGranted
      , include authorizationDenied
    ]

```

The authorization request and its two possible continuations form one production. At a more abstract rendering level, the entire production can be shown as one node; expanding it reveals authentication and the explicitly bracketed choice. The abstract contract makes both decisions possible. A product model may later attach an estimated or measured probability distribution without changing the set of legal traces.

5.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code      <- imported AuthCode "code"
    use resource  <- imported Resource "resource"
    let token     <- declare AccessToken "accessToken"
    let profile   <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token itself is visible only at the endpoints where the abstract protocol says it is transmitted. The grammar makes no claim about the authorization server's database schema or the application's session representation.

5.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The two named productions provide the first abstraction level. A renderer may collapse either production to a single node or expand it to its messages and nested alternatives. The

after line is a guarded nonterminal reference; the denial branch ends without exchanging a code.

5.6 Interaction diagrams by scenario

5.7 Requirements as observable constraints

```
def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
    (on "authorization callback" (previouslySame "state"
      "authorization requested"))

    , forbid "code exchange after denial"
    (previously "authorization denied" && event "token requested")

    , require "issued token came from a code"
    (on "access token issued" (previouslySame "code" "token requested"))

    , ctl "every decision reaches the application"
    (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

    , ctl "a protected result is never returned without a token request"
    (AG (resourceReturned --> previously tokenRequested))
  ]
```

5.7.1 Password secrecy

Observable events are not the same as universally readable plaintext. Every actor may observe that credential bytes were sent; only actors within the trusted user-agent boundary and authorization service may derive the password value. The theorem must name both the observer and its assumptions.

```
theorem oauth_password_secret
  (trace : Trace WireEvent)
  (accepted : (oauthSystem resource password).Accepts trace)
  (initial : InitiallyKnown password [.browser, .authorizationServer])
  (honest : Uncompromised [.browser, .authorizationServer])
  (protection : PerfectProtection protectFor) :
  not (Derives (knowledgeOf .networkAttacker trace) (secret password)) := by
  exact protectedFieldNotDerivable accepted initial honest protection
```

This statement deliberately permits the trusted browser boundary and authorization service to know the password. A theorem claiming secrecy from the browser would be false for a password typed into that browser. If the desired boundary is narrower, the actor model must represent a trusted credential component separately.

The theorem is also conditional. A compromised endpoint, logging side channel, or failed concrete protection refinement invalidates an assumption rather than being hidden by the grammar.

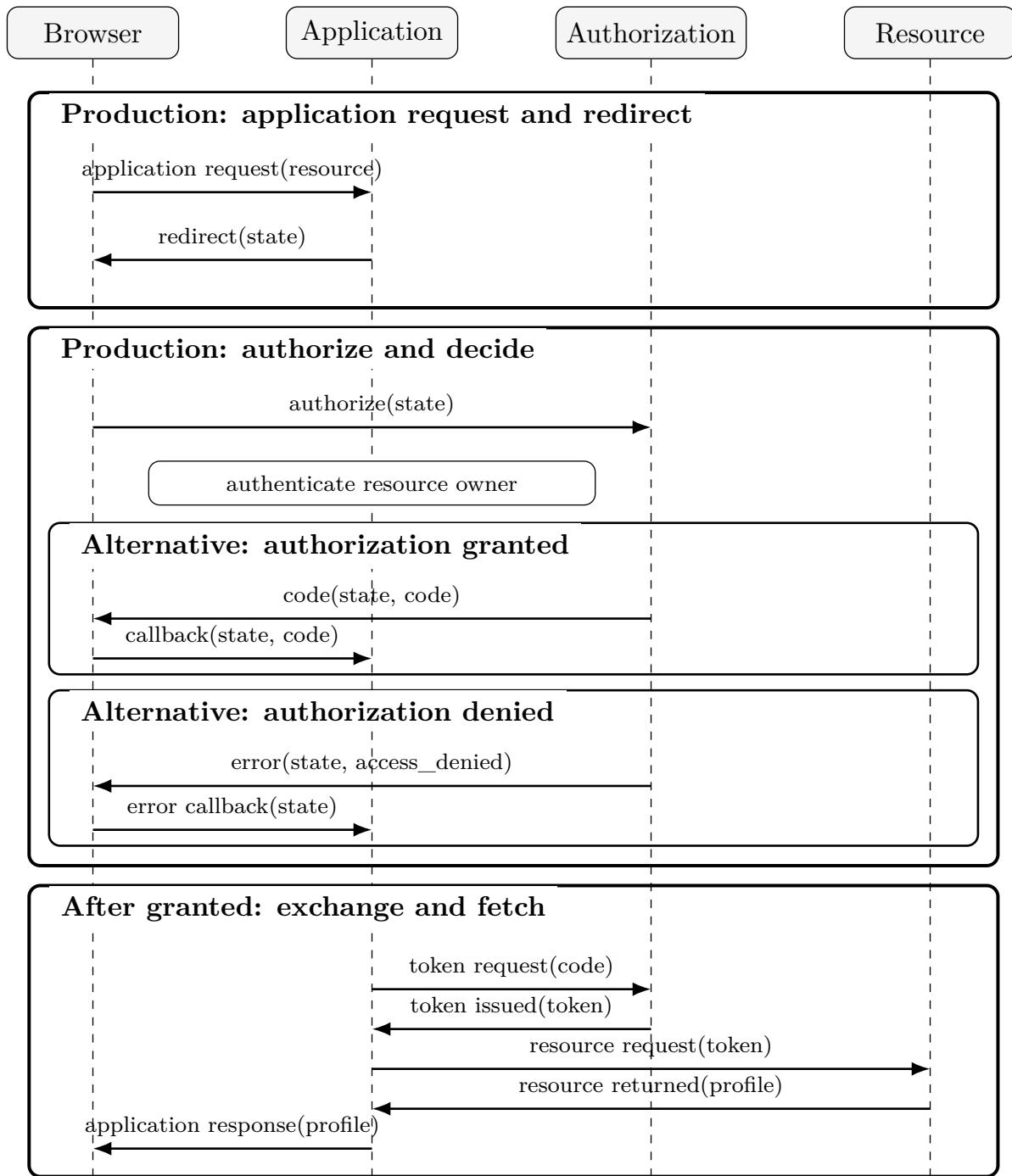


Figure 5.1: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

5.7.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
      , namedBytes "state" (encode state)
      , namedBytes "redirectUri" (encode callback)
    ]
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

This fixes framing, tags, and value relationships at an abstract byte-algebra level. A subsequent HTTP mapping might put those bytes into form fields; a protobuf mapping might use numbered fields. Either concrete alphabet must prove that decoding and re-encoding preserve the abstract terminal and its parameter bindings.

The use of `previously` is deliberately a trace predicate in this first version. It can later receive dedicated past-time syntax without changing the event model.

5.8 Probabilities and charts

The authorization service need not be random. The user supplies random input by granting or denying consent; the browser, queues, transport, and authorization service then react deterministically to the input they receive:

```
inductive UserDecision where
  | grant | deny

def userDecision : Choice UserDecision :=
  choose .user "consent decision"
    [ weighted (87 / 100) .grant
      , weighted (13 / 100) .deny ]

def authorizationServer
  (input : UserDecision) : Scenario :=
  match input with
  | .grant => authorizationGranted
  | .deny  => authorizationDenied

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
    source events; groupBy terminalOutcome; value count
  , line "end-to-end authorization latency" do
    source (correlate events by "state")
    x clock; y (elapsed "authorization requested" "application response")
  ]
```

After the user's choice, every transition may have probability one and the global model is still probabilistic. The grant branch carries probability mass 0.87 through the user's outbound queue, transport, the server's inbound queue, and the deterministic grant response. The denial branch similarly carries mass 0.13. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \quad P(\text{denial callback}) = 0.13.$$

The factors of one are deterministic enqueue, transport, and server steps. Their dwell times still affect the latency distribution even though they do not split the probability mass.

With queue scheduling and other unresolved choices included, the complete system is therefore a Markov decision process. Probabilistic edges describe inputs or measured outcomes; nondeterministic edges describe decisions not assigned a distribution, such as which enabled actor is scheduled next. An analysis first fixes or quantifies over a scheduling policy and then propagates the user-originated mass through the resulting paths.

For several sessions in flight, the MDP state contains every actor's bounded queues and its per-(`session`, `task`) residuals. One user's random event can change later queue pressure and scheduling possibilities for other sessions, so the probabilistic state is the composed system state rather than a special marker attached only to the user actor.

The percentages remain model inputs or measurements, not truths inferred from the grammar. The grammar defines which behaviours are possible; the probabilistic refinement says how mass is distributed over them.

5.9 Residual state machine

This automaton is intentionally behavioural. It does not contain browser cookies, database rows, server threads, or cryptographic library state. Those details may be essential to an implementation proof, but they are refinements of this contract, not the contract itself.

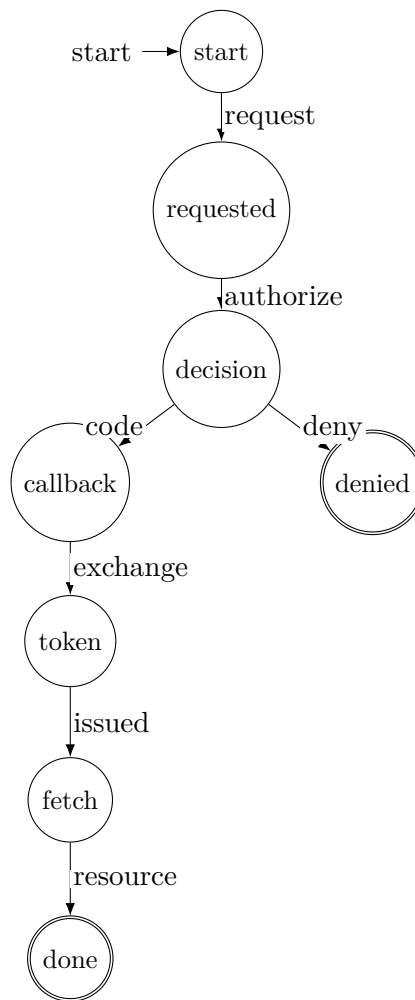


Figure 5.2: OAuth state machine derived from legal grammar continuations.

Chapter 6

CTL over Generated Futures

A grammar generates a branching transition system. CTL speaks directly about that branching: path quantifiers distinguish *there exists a future* from *every future*, while temporal operators distinguish the next state, eventuality, invariance, and until.

Formula	Reading	Typical use
$EF\ p$	some future can reach p	possibility / witness
$AF\ p$	every future eventually reaches p	termination / response
$EG\ p$	some path preserves p forever	viable invariant path
$AG\ p$	every reachable state satisfies p	safety invariant
$A[p\ U\ q]$	every path keeps p until q	constrained progress

For a request event carrying identifier r , an expected response property might say that on every continuation a matching success or failure eventually occurs. Authentication might say that every successful use of a session has an earlier issuance event with the same session identifier.

6.1 Past claims

CTL is future-facing at the current state, but security and protocol claims often refer to evidence in the past: “this capability was previously issued” or “this response matches an earlier request.” Since states are trace prefixes, past facts can initially be expressed as predicates over the accumulated trace. This gives a clear baseline without prematurely committing the surface language to a particular past-time temporal logic.

There is still an open design question. Explicit past operators may make properties more readable and allow better checking algorithms. The project should compare trace predicates, CTL with past operators, and compilation of finite-history monitors into state. The criterion is maintainability of specifications as the event schema evolves.

6.2 Fairness and time bounds

AF does not by itself distinguish a participant that is perpetually denied a turn from one that violates the protocol. Fairness assumptions must therefore be named, scoped, and visible in

theorem statements. Similarly, “eventually” is not “within 200 milliseconds.” Quantitative bounds belong in an explicit timed layer rather than being smuggled into timestamps and informal prose.

Chapter 7

Lean as the Rendered Language

The user-facing notation should render to a conservative subset of Lean. There is no second opaque specification file: domain declarations are Lean definitions, expected behaviours are Lean values, and requirements are Lean propositions or checked queries.

```
def checkout : Grammar CheckoutEvent :=
  request >>> (accepted <||> rejected)

def eventuallyAnswered : CTL Observation :=
  CTL.ag (CTL.implies requestPending
    (CTL.af (CTL.or requestAccepted requestRejected)))
```

Most semantic machinery should be imported from a fixed library. A system author normally supplies only:

- typed actors, identifiers, event kinds, and payloads;
- parameterised grammar definitions;
- projections from trace prefixes to observable propositions;
- named temporal requirements and proofs or decidable checks.

This boundary matters for automation. Generated Lean remains small enough to review, while library proofs and algorithms are reused rather than regenerated.

7.1 From specification to service

Lean can host executable code, including a small web server. That makes the formal artifact part of the running project rather than a document beside it. The same typed definitions can drive validation endpoints, test-trace generation, protocol schemas, visualisations, and runtime monitors. Extraction and integration choices must preserve the distinction between a verified behavioural boundary and an unverified infrastructure layer.

Chapter 8

Research and Implementation Roadmap

The project will proceed through small executable examples rather than attempting a complete logic and compiler at once.

8.1 Milestones

1. Stabilise the event envelope, including identity, causal links, actor endpoints, kind, payload, and a disciplined account of time.
2. Give the grammar algebra a trace semantics and establish laws for sequence, choice, guarded context, and interleaving.
3. Define parameter binding and participant projection, then test both on authentication and ordinary web request protocols.
4. Preserve named scenario boundaries while reading their references into one grammar; render interaction diagrams and residual state machines per scenario.
5. Treat independent requirements as trace-language intersections and report minimal inconsistent requirement sets.
6. Generate finite transition systems and evaluate the CTL fragment already represented by the LeanFM library.
7. Compare past-trace predicates with explicit past operators.
8. Render declarative pie and line charts as reductions over observable events.
9. Connect specifications to a Lean web service and generated protocol schemas, preserving one source of typed domain truth.

8.2 Proof obligations

The initial metatheory should make the following questions precise:

- Does a projected local behaviour faithfully implement its part of the global grammar?
- Under what finiteness restrictions is CTL checking decidable and executable?
- Which guards can be compiled into finite monitor state?
- When does modular composition preserve an established temporal property?
- Which fairness and timing assumptions are required by a liveness theorem?

The intended outcome is not merely a verifier. It is a development loop in which writing the behaviour is the fastest way to obtain working, reviewable software.

Part II

Security Protocol Exercises from the CSP Approach

Chapter 9

From CSP Roles to Multiparty Grammars

This part revisits examples from Ryan et al. [5] and its official Casper companion scripts. The goal is not to transliterate CSP operators. It is to use well-known security protocols as demanding exercises for the grammar-based method: multiple roles, fresh values, encrypted components, an active intruder, secrecy, agreement, and messages whose legality depends on the preceding trace.

9.1 Can actors really be specified separately?

Yes, provided composition is defined as synchronization over the same observable events rather than as concatenation of independent scripts. Each actor accepts a language over its own projection of the global event alphabet.

```
structure RoleSpec where
  actor      : Actor
  accepts    : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor

def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t , let $t|_a$ retain events sent or received by actor a . The composition accepts exactly when every local role accepts its view:

$$L(\text{composeRoles}(R_1, \dots, R_n)) = \{t \mid \forall i, t|_{a_i} \in L(R_i)\}.$$

One global message is therefore not duplicated into unrelated send and receive events. It has a single identity, byte string, sender, and recipient; both endpoint grammars constrain it. An unmatched send, an unexpected receive, or disagreement about bytes makes the intersection empty at that trace prefix.

9.1.1 Local state without global implementation state

A role may bind parameters from the events it observes. Those bindings are a parser environment for the trace, not a claim about implementation memory. A role can say “the nonce in this reply equals the nonce I previously sent” without specifying a table, variable layout, or retry mechanism.

9.2 A transport-neutral byte algebra

Security claims need more than message names. They need an algebra that says which bytes expose which values. We still need not choose HTTP, protobuf, ASN.1, or a particular cipher suite.

```
abbrev Bytes := ByteArray

inductive WireTerm where
| atom      : Name -> Bytes -> WireTerm
| tuple     : List WireTerm -> WireTerm
| encrypt   : KeyRef -> WireTerm -> WireTerm
| sign      : KeyRef -> WireTerm -> WireTerm

structure WireEvent where
  clock : ClockTimestamp
  id     : EventId
  priorIds : List EventId
  session : SessionId
  task    : TaskId
  from    : Actor
  to      : Actor
  term    : WireTerm
  bytes   : Bytes
  encodedBy : bytes = encodeCanonical term
```

At this level, encryption is a symbolic constructor with an encoding into bytes. The intruder may concatenate, split visible tuples, encrypt with known keys, and decrypt only with a derivable inverse key. A concrete refinement must replace this idealized algebra with a transport and cryptographic implementation whose security assumptions justify the abstraction.

9.3 Yahalom as three separate actors

Yahalom is an excellent first exercise because an initiator, responder, and trusted server exchange fresh nonces and receive a session key. The companion model states secrecy for the session key and agreement claims involving the nonces and key.

Let K_A and K_B be long-term keys shared by the server with Alice and Bob, N_A and N_B fresh nonces, and K_{AB} the proposed session key. The following actor grammars use symbolic wire terms whose canonical encodings are byte strings.

9.3.1 Alice, the initiator

```

def aliceRole (alice bob server : Actor) (ka : SharedKey) : RoleSpec :=
  role alice "Yahalom initiator" do
    let na <- fresh Nonce "na"

    from_ alice; to_ bob
    send (atom "initiator nonce" na.bytes)

    from_ server; to_ alice
    receive (encrypt ka.ref
      (tuple [actor bob, sessionKey "kab", nonce na, nonce "nb"]))
      where_ field "na" == na
    let kab <- take SessionKey "kab"
    let nb <- take Nonce "nb"
    let ticketForBob <- take WireTerm "ticketForBob"

    from_ alice; to_ bob
    send ticketForBob
    send (encrypt kab.ref (nonce nb))

```

Alice's `where_` clause is the context-sensitive step: the received nonce must equal the value generated earlier. The surface structure is context-free, but legality depends on the binding environment accumulated from the prefix.

9.3.2 Bob, the responder

```

def bobRole (alice bob server : Actor) (kb : SharedKey) : RoleSpec :=
  role bob "Yahalom responder" do
    from_ alice; to_ bob
    receive (atom "initiator nonce" ?naBytes)
    let na <- take Nonce "na"
    let nb <- fresh Nonce "nb"

    from_ bob; to_ server
    send (encrypt kb.ref
      (tuple [actor alice, nonce na, nonce nb]))

    from_ alice; to_ bob
    receive (encrypt kb.ref (tuple [actor alice, sessionKey "kab"]))
    let kab <- take SessionKey "kab"
    receive (encrypt kab.ref (nonce nb)) where_ field "nb" == nb

```

Bob need not know Alice's local control state. He constrains only the events visible at his boundary and the values learned from them.

9.3.3 The trusted server

```

def serverRole (alice bob server : Actor)
  (ka kb : SharedKey) : RoleSpec :=
  role server "Yahalom key server" do
    from_ bob; to_ server

```

```

receive (encrypt kb.ref
  (tuple [actor alice, nonce "na", nonce "nb"]))
let na <- take Nonce "na"
let nb <- take Nonce "nb"
let kab <- fresh SessionKey "kab"

let bobTicket := encrypt kb.ref
  (tuple [actor alice, sessionKey kab])
from_ server; to_ alice
sendMany
  [ encrypt ka.ref
    (tuple [actor bob, sessionKey kab, nonce na, nonce nb])
  , bobTicket
  ]

```

9.3.4 Global assembly and context conditions

```

def yahalom : GlobalGrammar :=
  composeRoles
    [ aliceRole .alice .bob .server keyAlice
    , bobRole .alice .bob .server keyBob
    , serverRole .alice .bob .server keyAlice keyBob
    , intruderRole .ivo
    ]
  |> where_ (distinct [.alice, .bob, .server, .ivo])
  |> where_ (freshValuesNeverRepeat ["na", "nb", "kab"])
  |> where_ (longTermKeysKnownOnlyToOwnersAndServer)

```

The three honest roles can be authored and checked separately. Composition exposes incompatibilities: for example, if Alice expects the server's first encrypted tuple to name Bob but the server grammar could name another responder, the shared global event cannot satisfy both projections.

9.4 The attacker is another grammar plus deduction

An active network attacker observes public wire events, blocks them, replays known bytes, and emits any term derivable from current knowledge. It does not learn the plaintext of an encryption merely because its encoded bytes are observable.

```

inductive Derives : Knowledge -> WireTerm -> Prop where
  | initial : term in knowledge -> Derives knowledge term
  | tuple   : (forall x in xs, Derives knowledge x) ->
    Derives knowledge (.tuple xs)
  | project : Derives knowledge (.tuple xs) -> x in xs ->
    Derives knowledge x
  | encrypt : Derives knowledge key -> Derives knowledge body ->
    Derives knowledge (.encrypt key body)
  | decrypt : Derives knowledge (.encrypt key body) ->
    Derives knowledge (inverse key) -> Derives knowledge body

```

```
def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe publicWireEvents
    mayBlock
    mayReplay
    maySend fun knowledge term => Derives knowledge term
```

This is the grammar analogue of the CSP/Dolev–Yao attacker: possible hostile messages are generated from observable history and constrained by a context-sensitive derivability relation.

9.5 Secrecy and agreement become theorems

A secrecy theorem quantifies over every accepted trace, not merely the happy-path scenario. For Yahalom:

```
theorem yahalom_session_key_secret
  (trace : Trace WireEvent)
  (accepted : yahalom.Accepts trace)
  (honest : Uncompromised [.alice, .bob, .server]) :
  not (Derives (knowledgeOf .ivo trace) (sessionKey accepted.kab)) := by
  -- proof uses the fixed symbolic-crypto library
  exact secrecyFromKeyOwnership accepted honest

theorem yahalom_bob_agrees_with_alice
  (trace : Trace WireEvent)
  (accepted : yahalom.Accepts trace)
  (bobDone : completed trace .bob) :
  exists run, peer run == .alice &&
    run.na == bobDone.na && run.nb == bobDone.nb := by
  exact agreementFromNonceChallenge accepted bobDone
```

The exact proof scripts depend on the library still to be built. The important design constraint is already visible: theorem statements refer only to accepted event traces, knowledge derived from those traces, and explicit compromise assumptions.

9.6 Otway–Rees: guards are not optional decoration

The Otway–Rees companion model introduces a run marker M and guards that the two participants differ. Each participant encrypts its nonce, marker, and actor names under its long-term server key. The server returns separate encrypted components.

This example adds three useful `where_` obligations:

```
where_ alice != bob
where_ response.field "marker" == request.field "marker"
where_ encryptedPart.names == [alice, bob]
```

The grammar shape remains a small sequence of messages. The accepted language is not context-free once equality of an unbounded marker and actor parameters matters. Keeping these tests as named guards preserves the readable protocol skeleton while making the extra context explicit.

The example also distinguishes ordinary secrecy from *strong secrecy*: an attacker should not be able to distinguish executions merely by substituting one candidate session key for another. That is a relational property over pairs of trace distributions, beyond a single CTL state formula, and should be represented as a separate theorem family.

9.7 Needham–Schroeder: public-key lookup as behaviour

The seven-message public-key example makes key discovery observable. Alice requests Bob’s public key from a server, receives a signed binding, sends an encrypted nonce and identity to Bob, and later answers Bob’s nonce. Bob performs the symmetric key lookup for Alice.

Actor separation is particularly valuable here:

- the key server grammar promises signed actor–public-key bindings;
- Alice’s grammar accepts Bob’s key only where the signature verifies and the named actor is Bob;
- Bob imposes the corresponding constraint for Alice;
- the nonce-response messages reuse previously bound values;
- the intruder grammar may substitute any derivable public key or replay any signed binding, but cannot manufacture a server signature.

```
receive (sign serverSigningKey (tuple [actor bob, publicKey "pkb"]))
  where_ verifiesWith serverPublicKey
  where_ field "actor" == bob

send (encrypt pkb (tuple [nonce na, actor alice]))

receive (encrypt aliceSecretView (tuple [nonce na, nonce "nb"]))
  where_ field "na" == na
```

This demonstrates why “all behaviour is observable” must not mean “all plaintext is public.” The globally recorded event exposes endpoints, timing, kind, and wire bytes. Which symbolic values an actor can recover from those bytes is governed by the deduction relation.

9.8 What these exercises demand from the library

The three case studies produce a concrete implementation checklist:

1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables, exports, imports, and prefix-dependent guards;
4. a transport-neutral term-to-byte relation;
5. explicit initial knowledge and compromise assumptions;

6. attacker closure under symbolic construction and decryption;
7. trace secrecy, agreement, and relational strong-secrecy theorem schemas;
8. counterexamples rendered simultaneously as a global interaction diagram and as each actor's local projection.

These are not security-specific embellishments. They are a rigorous test of whether multi-party context-sensitive grammars can remain modular while expressing the information flow on which real behavioural claims depend.

Bibliography

- [1] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.
- [2] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL <https://github.com/rfielding/leanFM>. Accessed 2026-09-21.
- [3] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-5. doi: 10.4324/9781315693361.
- [4] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.
- [5] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

Index

- agreement, 36
- alphabet
 - vague, 9
- bytes, 33
- chart specification, 13
- composition
 - requirement intersection, 12
- CSP, 32
- CTL, 26
- event, 4
- formal methods, 2
- grammar
 - context-free, 6
 - multiparty, 6
- intruder model, 35
- Lean, 28
- Needham–Schroeder protocol, 37
- OAuth, 17
- observable event, 8
- Otway–Rees protocol, 36
- parameter flow, 6
- password secrecy, 21
- past-time logic, 26
- probability, 10
- protocol role, 32
- scenario, 8
- secrecy, 36
- temporal logic, 26
- time, 5
- trace, 4
- Yahalom protocol, 33